Resolución — Taller SQL sobre HR 6 — Funciones de ventana

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Tres formas de numerar a los empleados

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Numerar a los empleados por salario de mayor a menor de tres maneras distintas (`ROW_NUMBER`, `RANK` y `DENSE_RANK`) y mostrar a los ocho mejor pagados.

Mostrar: nombre completo, salario, número de fila, posición y posición densa, ordenados por número de fila.

Idea. Una función de ventana calcula sobre un grupo de filas sin colapsarlas: cada empleado conserva su fila y recibe un valor calculado. `OVER (ORDER BY ...)` define el orden de la ventana.

Pasos.

1. `ROW_NUMBER()` numera las filas 1, 2, 3... sin repetir, incluso con empates.
2. `RANK()` repite la posición cuando hay empate y salta los números que siguen.
3. `DENSE_RANK()` repite la posición y no salta.
4. Una función de ventana no puede usarse en el `WHERE`, así que se calcula en una vista en línea y se filtra afuera con `x.fila <= 8`.

```
SELECT x.nombre,
       x.salary,
       x.fila,
       x.posicion,
       x.posicion_densa
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre,
             e.salary,
             ROW_NUMBER() OVER (ORDER BY e.salary DESC, e.last_name)
               AS fila,
             RANK() OVER (ORDER BY e.salary DESC) AS posicion,
             DENSE_RANK() OVER (ORDER BY e.salary DESC) AS posicion_densa
      FROM hr.employees e) x
WHERE x.fila <= 8
ORDER BY x.fila;
```

Resultado en HR. 8 filas y 5 columnas:

```
NOMBRE            SALARY  FILA  POSICION  POSICION_DENSA
----------------  ------  ----  --------  --------------
Steven King        24000     1         1               1
Lex Garcia         17000     2         2               2
Neena Yang         17000     3         2               2
John Singh         14000     4         4               3
Karen Partners     13500     5         5               4
Michael Martinez   13000     6         6               5
Nancy Gruenberg    12008     7         7               6
Shelley Higgins    12008     8         7               6
```

Ojo. Escribir `RANK() OVER (...) <= 3` directo en el `WHERE` falla con `ORA-30483` (las funciones de ventana no se permiten ahí): por eso la vista en línea. Con los dos empates de HR se ve la diferencia: `RANK` da 1, 2, 2, 4 y `DENSE_RANK` da 1, 2, 2, 3. `ROW_NUMBER` rompe el empate sin criterio propio, por eso se le añade `e.last_name`.

Ejercicio 2 — Los tres mejor pagados de cada departamento

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Mostrar, para cada departamento, a los tres empleados con mayor salario. Si hay un empate en el tercer lugar, deben salir todos los empatados.

Mostrar: departamento, nombre completo, salario y posición dentro del departamento, ordenados por departamento, posición y nombre.

Idea. `PARTITION BY` divide las filas en grupos y la función se calcula dentro de cada grupo, como un `GROUP BY` que no colapsa las filas.

Pasos.

1. `RANK() OVER (PARTITION BY d.department_id ORDER BY e.salary DESC)` numera a los empleados dentro de su departamento.
2. Se une `hr.departments` para mostrar el nombre del departamento.
3. La vista en línea calcula la posición y la consulta externa se queda con `x.posicion <= 3`.
4. Se ordena por departamento, posición y nombre.

```
SELECT x.department_name,
       x.nombre,
       x.salary,
       x.posicion
FROM (SELECT d.department_name,
             e.first_name || ' ' || e.last_name AS nombre,
             e.salary,
             RANK() OVER (PARTITION BY d.department_id
                          ORDER BY e.salary DESC) AS posicion
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id) x
WHERE x.posicion <= 3
ORDER BY x.department_name, x.posicion, x.nombre;
```

Resultado en HR. 26 filas y 4 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  NOMBRE           SALARY  POSICION
---------------  ---------------  ------  --------
Accounting       Shelley Higgins   12008         1
Accounting       William Gietz      8300         2
Administration   Jennifer Whalen    4400         1
Executive        Steven King       24000         1
Executive        Lex Garcia        17000         2
Executive        Neena Yang        17000         2
```

Ojo. Con RANK los empates comparten posición, y por eso IT muestra cuatro filas: David Williams y Valli Jackson empatan en el tercer lugar con 4800. Si se necesitaran exactamente tres filas por departamento, se usaría ROW_NUMBER con un criterio de desempate.

Ejercicio 3 — Mejor pagados que su departamento, sin subconsulta

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar a los empleados cuyo salario es mayor que el promedio de su departamento, con ese promedio, resolviéndolo con una función de ventana.

Mostrar: nombre completo, salario y promedio del departamento (dos decimales), ordenados por departamento, salario descendente y apellido.

Idea. `AVG(...) OVER (PARTITION BY ...)` pone en cada fila el promedio de su grupo, sin colapsarlas. Así cada salario se compara con su promedio sin subconsulta.

Pasos.

1. `AVG(e.salary) OVER (PARTITION BY e.department_id)` agrega a cada empleado el promedio de su departamento.
2. Como la ventana no puede ir en el `WHERE`, se calcula en una vista en línea.
3. La consulta externa conserva las filas con `x.salary > x.promedio`.
4. `ROUND` se aplica solo al mostrar, para no comparar contra un valor redondeado.

```
SELECT x.nombre_empleado,
       x.salary,
       ROUND(x.promedio, 2) AS promedio_departamento
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre_empleado,
             e.last_name,
             e.department_id,
             e.salary,
             AVG(e.salary) OVER (PARTITION BY e.department_id) AS promedio
      FROM hr.employees e) x
WHERE x.salary > x.promedio
ORDER BY x.department_id, x.salary DESC, x.last_name;
```

Resultado en HR. 38 filas y 3 columnas; se muestran las primeras 6 filas:

```
NOMBRE_EMPLEADO   SALARY  PROMEDIO_DEPARTAMENTO
---------------  ------  --------------------
Michael Martinez   13000                  9500
Den Li             11000                  4150
Adam Fripp          8200               3475.56
Matthew Weiss       8000               3475.56
Payam Kaufling      7900               3475.56
Shanta Vollman      6500               3475.56
```

Ojo. Devuelve las mismas 38 personas y en el mismo orden que el ejercicio 2 del taller 3, resuelto con vista en línea y con subconsulta correlacionada. Aquí el promedio llega ya calculado en cada fila. Kimberely Grant, sin departamento, forma su propio grupo y no supera su propio promedio.

Ejercicio 4 — Cuánto pesa cada uno en la nómina de su departamento

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Para los empleados de Finance, mostrar qué porcentaje de la nómina del departamento representa el salario de cada uno.

Mostrar: nombre completo, salario, nómina del departamento y porcentaje con un decimal, de mayor a menor salario y, si empatan, por apellido.

Idea. `RATIO_TO_REPORT(x) OVER (...)` devuelve la fracción que x representa del total de su ventana. Equivale a `x / SUM(x) OVER (...)`.

Pasos.

1. `SUM(e.salary) OVER (PARTITION BY e.department_id)` da la nómina de todo el departamento en cada fila.
2. `RATIO_TO_REPORT(e.salary) OVER (PARTITION BY e.department_id)` da la fracción de cada salario.
3. Se multiplica por 100 y se redondea a un decimal.
4. El filtro por departamento va en el `WHERE`; se ordena por salario descendente y apellido.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       e.salary,
       SUM(e.salary) OVER (PARTITION BY e.department_id) AS nomina_depto,
       ROUND(100 * RATIO_TO_REPORT(e.salary)
                   OVER (PARTITION BY e.department_id), 1) AS porcentaje
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE d.department_name = 'Finance'
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 6 filas y 4 columnas:

```
NOMBRE             SALARY  NOMINA_DEPTO  PORCENTAJE
-----------------  ------  ------------  ----------
Nancy Gruenberg     12008         51608        23.3
Daniel Faviet        9000         51608        17.4
John Chen            8200         51608        15.9
Jose Manuel Urman    7800         51608        15.1
Ismael Sciarra       7700         51608        14.9
Luis Popp            6900         51608        13.4
```

Ojo. El `WHERE` se ejecuta antes que la ventana: si se filtrara por salario, el total del departamento ya no sería el real. Aquí el filtro es por el departamento completo y no distorsiona nada. Los porcentajes redondeados suman 100.0.

Variante. La misma cuenta escrita a mano, dividiendo el salario entre la suma de la ventana, da el mismo resultado:

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       e.salary,
       SUM(e.salary) OVER (PARTITION BY e.department_id) AS nomina_depto,
       ROUND(100 * e.salary
             / SUM(e.salary) OVER (PARTITION BY e.department_id), 1)
         AS porcentaje
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE d.department_name = 'Finance'
ORDER BY e.salary DESC, e.last_name;
```

Ejercicio 5 — Nómina acumulada por fecha de contratación

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar cómo creció la nómina de la empresa a medida que se contrató gente: para cada empleado, la nómina acumulada hasta su contratación. Mostrar solo las diez primeras contrataciones.

Mostrar: nombre completo, fecha de contratación (DD/MM/AAAA), salario y nómina acumulada, en orden de contratación y, si empatan, por identificador.

Idea. `SUM(...) OVER (ORDER BY ...)` con una ventana que va desde la primera fila hasta la fila actual da una suma acumulada: cada fila suma su salario al de todas las anteriores.

Pasos.

1. El orden de la ventana es `e.hire_date, e.employee_id`: la fecha manda y el identificador desempata.
2. `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` pide la suma desde la primera fila hasta la actual.
3. `ORDER BY` y `FETCH FIRST 10 ROWS ONLY` se aplican después de calcular la ventana, así que no hace falta una vista en línea.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS contratacion,
       e.salary,
       SUM(e.salary) OVER (ORDER BY e.hire_date, e.employee_id
                           ROWS BETWEEN UNBOUNDED PRECEDING
                                    AND CURRENT ROW) AS acumulado
FROM hr.employees e
ORDER BY e.hire_date, e.employee_id
FETCH FIRST 10 ROWS ONLY;
```

Resultado en HR. 10 filas y 4 columnas; se muestran las primeras 6 filas:

```
NOMBRE           CONTRATACION  SALARY  ACUMULADO
---------------  ------------  ------  ---------
Lex Garcia       13/01/2011     17000      17000
Susan Jacobs     07/06/2012      6500      23500
Hermann Brown    07/06/2012     10000      33500
Shelley Higgins  07/06/2012     12008      45508
William Gietz    07/06/2012      8300      53808
Daniel Faviet    16/08/2012      9000      62808
```

Ojo. Si se omite el marco, el valor por defecto con `ORDER BY` es `RANGE`, que agrupa las filas con el mismo valor de ordenamiento: las cuatro personas contratadas el 07/06/2012 recibirían todas el mismo acumulado (53808) en lugar de ir creciendo una a una. Con `ROWS` cada fila suma la suya.

Ejercicio 6 — Días entre una contratación y la anterior

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Calcular cuántos días pasaron entre cada contratación y la anterior, en orden de fecha, y mostrar las cinco pausas más largas.

Mostrar: nombre completo, fecha de contratación, fecha de la contratación anterior (DD/MM/AAAA) y días de pausa, de mayor a menor pausa y, si empatan, por fecha.

Idea. `LAG(col)` trae el valor de la fila anterior en el orden de la ventana (y `LEAD`, el de la siguiente). Restar dos fechas da la diferencia en días.

Pasos.

1. `LAG(e.hire_date) OVER (ORDER BY e.hire_date, e.employee_id)` trae la fecha de la contratación anterior.
2. La primera fila no tiene anterior: `LAG` devuelve `NULL` y se descarta con `WHERE x.anterior IS NOT NULL`.
3. Los días son `x.hire_date - x.anterior`.
4. Se ordena por días descendente y `FETCH FIRST 5 ROWS ONLY` deja las cinco mayores.

```
SELECT x.nombre,
       TO_CHAR(x.hire_date, 'DD/MM/YYYY') AS contratacion,
       TO_CHAR(x.anterior, 'DD/MM/YYYY') AS contratacion_anterior,
       x.hire_date - x.anterior AS dias
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre,
             e.hire_date,
             LAG(e.hire_date) OVER (ORDER BY e.hire_date, e.employee_id)
               AS anterior
      FROM hr.employees e) x
WHERE x.anterior IS NOT NULL
ORDER BY dias DESC, x.hire_date
FETCH FIRST 5 ROWS ONLY;
```

Resultado en HR. 5 filas y 4 columnas:

```
NOMBRE            CONTRATACION  CONTRATACION_ANTERIOR  DIAS
----------------  ------------  ---------------------  ----
Susan Jacobs      07/06/2012    13/01/2011              511
Payam Kaufling    01/05/2013    07/12/2012              145
Den Li            07/12/2012    17/08/2012              112
Nandita Sarchand  27/01/2014    17/10/2013              102
Karen Partners    05/01/2015    01/10/2014               96
```

Ojo. Cuando varias personas entran el mismo día, la pausa entre ellas es 0, y el desempate por `employee_id` dentro de la ventana fija cuál es la anterior de cuál. La pausa más larga es de 511 días y la protagoniza Susan Jacobs: es la primera de las contrataciones del 07/06/2012 y su anterior es Lex Garcia, de enero de 2011.

Ejercicio 7 — La distancia al siguiente salario con LEAD

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Para los empleados de Finance, ordenados de mayor a menor salario, mostrar cuánto más gana cada uno que la persona que le sigue en la lista.

Mostrar: nombre completo, salario, salario de la persona siguiente y diferencia, de mayor a menor salario y, si empatan, por apellido. El último no tiene siguiente.

Idea. `LEAD(col, n, valor_por_defecto)` mira n filas hacia adelante (una por defecto). Cuando no hay fila siguiente devuelve `NULL`, o el valor por defecto si se indica.

Pasos.

1. `LEAD(e.salary) OVER (ORDER BY e.salary DESC, e.last_name)` trae el salario de la persona que sigue.
2. La diferencia es `e.salary` menos ese salario.
3. Se filtra Finance con el `JOIN` a `hr.departments` y el `WHERE`.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       e.salary,
       LEAD(e.salary) OVER (ORDER BY e.salary DESC, e.last_name)
         AS salario_siguiente,
       e.salary - LEAD(e.salary) OVER (ORDER BY e.salary DESC, e.last_name)
         AS diferencia
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE d.department_name = 'Finance'
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 6 filas y 4 columnas:

```
NOMBRE             SALARY  SALARIO_SIGUIENTE  DIFERENCIA
-----------------  ------  -----------------  ----------
Nancy Gruenberg     12008               9000        3008
Daniel Faviet        9000               8200         800
John Chen            8200               7800         400
Jose Manuel Urman    7800               7700         100
Ismael Sciarra       7700               6900         800
Luis Popp            6900             (null)      (null)
```

Ojo. El último de la lista no tiene siguiente y queda con `NULL`; con `LEAD(e.salary, 1, 0)` se pondría un 0 en su lugar. El `ORDER BY` de la ventana y el de la consulta son independientes: el de la ventana define quién es el siguiente; el de la consulta, cómo se muestran las filas.

Ejercicio 8 — Cuartiles de salario con NTILE

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Dividir a todos los empleados en cuatro grupos del mismo tamaño según su salario (cuartiles) y mostrar, para cada cuartil, cuántas personas tiene y los salarios mínimo y máximo.

Mostrar: número de cuartil, cantidad de empleados, salario mínimo y salario máximo, ordenados por cuartil.

Idea. `NTILE(n)` reparte las filas ordenadas en n grupos del mismo tamaño y numera cada fila con el grupo al que cae. Después se agrupa por ese número.

Pasos.

1. `NTILE(4) OVER (ORDER BY e.salary)` asigna el cuartil 1 a los salarios más bajos y el 4 a los más altos.
2. Una vista en línea calcula el cuartil de cada empleado.
3. La consulta externa agrupa por cuartil y calcula `COUNT(*)`, `MIN` y `MAX`.

```
SELECT x.cuartil,
       COUNT(*) AS empleados,
       MIN(x.salary) AS minimo,
       MAX(x.salary) AS maximo
FROM (SELECT e.salary,
             NTILE(4) OVER (ORDER BY e.salary) AS cuartil
      FROM hr.employees e) x
GROUP BY x.cuartil
ORDER BY x.cuartil;
```

Resultado en HR. 4 filas y 4 columnas:

```
CUARTIL  EMPLEADOS  MINIMO  MAXIMO
-------  ---------  ------  ------
      1         27    2100    3100
      2         27    3100    6200
      3         27    6200    9000
      4         26    9000   24000
```

Ojo. `NTILE` reparte por posición y no por valor. Con 107 empleados los grupos salen de 27, 27, 27 y 26 filas, y personas con el mismo salario pueden quedar en cuartiles distintos: en HR hay salarios repartidos entre dos cuartiles.

Ejercicio 9 — El mejor pagado del departamento y la brecha

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Para los empleados de IT (`department_id` 60), mostrar el nombre de la persona mejor pagada del departamento y cuánto menos gana cada uno respecto a ella.

Mostrar: nombre completo, salario, nombre del mejor pagado y brecha, de mayor a menor salario y, si empatan, por apellido.

Idea. `FIRST_VALUE(x) OVER (PARTITION BY g ORDER BY o)` devuelve el valor de x en la primera fila de la ventana. Con orden descendente por salario, es el del mejor pagado.

Pasos.

1. La ventana es `PARTITION BY e.department_id ORDER BY e.salary DESC, e.last_name`.
2. `FIRST_VALUE` del nombre da el mejor pagado y `FIRST_VALUE` del salario da su sueldo.
3. La brecha es ese sueldo menos el de la fila.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       e.salary,
       FIRST_VALUE(e.first_name || ' ' || e.last_name)
         OVER (PARTITION BY e.department_id
               ORDER BY e.salary DESC, e.last_name) AS mejor_pagado,
       FIRST_VALUE(e.salary)
         OVER (PARTITION BY e.department_id
               ORDER BY e.salary DESC, e.last_name) - e.salary AS brecha
FROM hr.employees e
WHERE e.department_id = 60
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 5 filas y 4 columnas:

```
NOMBRE           SALARY  MEJOR_PAGADO     BRECHA
---------------  ------  ---------------  ------
Alexander James    9000  Alexander James       0
Bruce Miller       6000  Alexander James    3000
Valli Jackson      4800  Alexander James    4200
David Williams     4800  Alexander James    4200
Diana Nguyen       4200  Alexander James    4800
```

Ojo. `LAST_VALUE` engaña: con `ORDER BY` el marco por defecto llega hasta la fila actual y no hasta el final, así que devuelve el salario de la propia fila. Para ver el último de verdad hay que escribir `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`; en IT eso da 4200 para todos.

Ejercicio 10 — Promedio móvil de tres contrataciones

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Calcular el promedio móvil del salario de las tres últimas contrataciones (la fila actual y las dos anteriores en orden de fecha) y mostrarlo para las personas contratadas en 2018.

Mostrar: nombre completo, fecha de contratación (DD/MM/AAAA), salario y promedio móvil con dos decimales, en orden de contratación y, si empatan, por identificador.

Idea. Una ventana con `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` abarca tres filas: la actual y las dos anteriores en el orden indicado. Es la base de un promedio móvil.

Pasos.

1. `AVG(e.salary) OVER (ORDER BY e.hire_date, e.employee_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)` calcula el promedio de las tres filas.
2. La ventana se calcula sobre toda la plantilla, en una vista en línea.
3. El filtro de 2018 va afuera, sobre el resultado ya calculado.

```
SELECT x.nombre,
       TO_CHAR(x.hire_date, 'DD/MM/YYYY') AS contratacion,
       x.salary,
       ROUND(x.promedio_movil, 2) AS promedio_movil
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre,
             e.employee_id,
             e.hire_date,
             e.salary,
             AVG(e.salary) OVER (ORDER BY e.hire_date, e.employee_id
                                 ROWS BETWEEN 2 PRECEDING
                                          AND CURRENT ROW) AS promedio_movil
      FROM hr.employees e) x
WHERE x.hire_date >= DATE '2018-01-01'
ORDER BY x.hire_date, x.employee_id;
```

Resultado en HR. 11 filas y 4 columnas; se muestran las primeras 6 filas:

```
NOMBRE            CONTRATACION  SALARY  PROMEDIO_MOVIL
----------------  ------------  ------  --------------
Charles Johnson   04/01/2018      6200            3700
Douglas Grant     13/01/2018      2600         3766.67
Mattea Marvins    24/01/2018      7200         5333.33
Eleni Zlotkey     29/01/2018     10500         6766.67
Girard Geoni      03/02/2018      2800         6833.33
Hazel Philtanker  06/02/2018      2200         5166.67
```

Ojo. El filtro de 2018 tiene que ir fuera de la vista en línea. Si se pusiera dentro, las contrataciones de 2017 desaparecerían antes de calcular y el primer promedio se haría con una sola fila: Charles Johnson saldría con 6200 en lugar de 3700, que es el promedio de él y de las dos personas que entraron justo antes.

Ejercicio 11 — El más antiguo de cada departamento

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Mostrar, para cada departamento, a la persona que lleva más tiempo: la que fue contratada primero.

Mostrar: departamento, nombre completo y fecha de contratación (DD/MM/AAAA), ordenados por nombre del departamento.

Idea. Quedarse con una fila por grupo es un patrón muy común: se numeran las filas de cada grupo con `ROW_NUMBER` en el orden que interesa y se conserva la número 1.

Pasos.

1. `ROW_NUMBER() OVER (PARTITION BY d.department_id ORDER BY e.hire_date, e.employee_id)` numera a cada persona dentro de su departamento, de la más antigua a la más reciente.
2. La vista en línea calcula el número de fila.
3. La consulta externa se queda con `x.fila = 1`.

```
SELECT x.department_name,
       x.nombre,
       TO_CHAR(x.hire_date, 'DD/MM/YYYY') AS contratacion
FROM (SELECT d.department_name,
             e.first_name || ' ' || e.last_name AS nombre,
             e.hire_date,
             ROW_NUMBER() OVER (PARTITION BY d.department_id
                                ORDER BY e.hire_date, e.employee_id) AS fila
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id) x
WHERE x.fila = 1
ORDER BY x.department_name;
```

Resultado en HR. 11 filas y 3 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  NOMBRE           CONTRATACION
---------------  ---------------  ------------
Accounting       Shelley Higgins  07/06/2012
Administration   Jennifer Whalen  17/09/2013
Executive        Lex Garcia       13/01/2011
Finance          Daniel Faviet    16/08/2012
Human Resources  Susan Jacobs     07/06/2012
IT               David Williams   25/06/2015
```

Ojo. Si dos personas empatan en la fecha, `ROW_NUMBER` se queda con una sola; por eso el orden de la ventana lleva `e.employee_id` como desempate. Si lo que se quiere es conservar a todas las empatadas, se usa `RANK`.

Variante. Con `RANK() ... = 1` salen todos los empatados. En Accounting, Shelley Higgins y William Gietz entraron el mismo día (07/06/2012), así que aparecen los dos y el resultado es de 12 filas en vez de 11:

```
SELECT x.department_name,
       x.nombre,
       TO_CHAR(x.hire_date, 'DD/MM/YYYY') AS contratacion
FROM (SELECT d.department_name,
             e.first_name || ' ' || e.last_name AS nombre,
             e.hire_date,
             RANK() OVER (PARTITION BY d.department_id
                          ORDER BY e.hire_date) AS posicion
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id) x
WHERE x.posicion = 1
ORDER BY x.department_name, x.nombre;
```

Ejercicio 12 — Departamentos por nómina y por promedio

Nivel: Reto · **Tablas:** `employees`, `departments`

Se pide. Mostrar, para cada departamento, su nómina total y su salario promedio, la posición del departamento según cada criterio y cuántos puestos cambia de uno a otro.

Mostrar: departamento, nómina, posición por nómina, promedio (dos decimales), posición por promedio y cambio (posición por nómina menos posición por promedio), ordenados por posición por nómina.

Idea. Las funciones de ventana se calculan después de `GROUP BY`, así que pueden ordenar sobre agregados como `SUM(e.salary)` o `AVG(e.salary)`.

Pasos.

1. El `GROUP BY` por departamento calcula `SUM` y `AVG`.
2. `RANK() OVER (ORDER BY SUM(e.salary) DESC)` ordena a los departamentos por nómina.
3. `RANK() OVER (ORDER BY AVG(e.salary) DESC)` los ordena por promedio.
4. El cambio es la resta de las dos posiciones, y se ordena por la primera.

```
SELECT d.department_name,
       SUM(e.salary) AS nomina,
       RANK() OVER (ORDER BY SUM(e.salary) DESC) AS pos_nomina,
       ROUND(AVG(e.salary), 2) AS promedio,
       RANK() OVER (ORDER BY AVG(e.salary) DESC) AS pos_promedio,
       RANK() OVER (ORDER BY SUM(e.salary) DESC)
         - RANK() OVER (ORDER BY AVG(e.salary) DESC) AS cambio
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY pos_nomina;
```

Resultado en HR. 11 filas y 6 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  NOMINA  POS_NOMINA  PROMEDIO  POS_PROMEDIO  CAMBIO
---------------  ------  ----------  --------  ------------  ------
Sales            304500           1   8955.88             5      -4
Shipping         156400           2   3475.56            11      -9
Executive         58000           3  19333.33             1       2
Finance           51608           4   8601.33             6      -2
IT                28800           5      5760             8      -3
Purchasing        24900           6      4150            10      -4
```

Ojo. Dentro de `OVER (ORDER BY ...)` se escribe la expresión agregada completa, no el alias. Un departamento puede ser alto por nómina y bajo por promedio: Shipping es el segundo por nómina, porque tiene 45 empleados, y el último por promedio.